Appendix C Selected OOP source code

This appendix prints the project code where object oriented concepts are directly visible. Each block is an unchanged extract from the documented source revision, with its file and line range. The complete compilable source remains in the repository. The separate OOP Code Extracts PDF contains these same pages for convenient printing.

C.1 Encapsulation and validation

Point keeps coordinates private. Construction and setters use one check, so an invalid coordinate cannot enter through an ordinary public edit. Accessors permit observation without exposing writable fields.

Source `include/scene.hpp` lines 24 to 40.

```
class Point {
    double x_ = 0;
    double y_ = 0;
    static double checked(double value) {
        if (!std::isfinite(value) || std::abs(value) > 1e7)
            throw std::invalid_argument("Coordinate must be finite and within +/-10000000.");
        return value;
    }

  public:
    Point() = default;
    Point(double x, double y) : x_(checked(x)), y_(checked(y)) {}
    double x() const noexcept { return x_; }
    void setX(double value) { x_ = checked(value); }
    double y() const noexcept { return y_; }
    void setY(double value) { y_ = checked(value); }
};
```

Scene applies the same principle to parameters. A successful radius edit advances the model revision, allowing dependent results and history to detect input changes.

Source `include/scene.hpp` lines 108 to 113.

```
    void setRadius(double value) {
        if (!std::isfinite(value) || value <= 0 || value > 1e7)
            throw std::invalid_argument("Radius must be positive and at most 10000000.");
        radius_ = value;
        touch();
    }
```

C.2 Abstraction and the Template Method

Algorithm is abstract because mode() and calculate() are pure virtual. Its public run() function defines the execution lifecycle, while protected calculate() is the specialization point. A virtual destructor permits safe destruction through an Algorithm pointer.

Source `include/algorithms.hpp` lines 66 to 74.

```
class Algorithm {
  protected:
    virtual Result calculate(const Scene &scene, bool record) const = 0;

  public:
    virtual ~Algorithm() = default;
    virtual Mode mode() const noexcept = 0;
    Result run(const Scene &scene, bool record = true) const;
};
```

Source `src/algorithms.cpp` lines 627 to 632.

```
Result Algorithm::run(const Scene &scene, bool record) const {
    Engine::validate(scene);
    Result result = calculate(scene, record);
    Engine::verify(scene, result);
    return result;
}
```

The call to calculate() dispatches to the concrete strategy at runtime. Engine validation and result checking stay in the shared lifecycle. The concrete hull class supplies the calculation and identifies its mode; override checks the signatures, and final closes this implementation to further inheritance.

Source `include/algorithms.hpp` lines 76 to 81.

```
class HullAlgorithm final : public Algorithm {
    Result calculate(const Scene &scene, bool record) const override;

  public:
    Mode mode() const noexcept override { return Mode::Hull; }
};
```

Template Method is a design pattern here. It does not mean that Algorithm is a C++ template. This distinction separates the object oriented lifecycle from generic programming syntax.

C.3 Runtime polymorphism and owned strategies

The application can call AlgorithmRunner without depending on Engine's registry. Engine implements that interface and owns concrete Algorithm objects through unique_ptr. Composition manages ownership; inheritance provides substitutable behaviour.

Source `include/algorithms.hpp` lines 146 to 161.

```
class AlgorithmRunner {
  public:
    virtual ~AlgorithmRunner() = default;
    virtual Result execute(const Scene &scene, Mode mode, bool record = true) const = 0;
};
// Owning registry of algorithm Strategies, keyed by scoped Mode values.
class Engine final : public AlgorithmRunner {
    std::map<Mode, std::unique_ptr<Algorithm>> algorithms_;

  public:
    Engine();
    Result execute(const Scene &scene, Mode mode, bool record = true) const override;
    static void validate(const Scene &scene);
    static bool verify(const Scene &scene, Result &result);
};
```

Source `src/algorithms.cpp` lines 634 to 657.

```
Engine::Engine() {
    auto add = [this](std::unique_ptr<Algorithm> algorithm) {
        const Mode key = algorithm->mode();
        algorithms_.emplace(key, std::move(algorithm));
    };
    add(std::make_unique<HullAlgorithm>());
    add(std::make_unique<GeofenceAlgorithm>());
    add(std::make_unique<DijkstraAlgorithm>());
    add(std::make_unique<AStarAlgorithm>());
    add(std::make_unique<DBSCANAlgorithm>(false));
    add(std::make_unique<ClosestPairAlgorithm>());
    add(std::make_unique<SpatialQueryAlgorithm>(Mode::KDNearest));
    add(std::make_unique<SpatialQueryAlgorithm>(Mode::KDRadius));
    add(std::make_unique<DBSCANAlgorithm>(true));
    add(std::make_unique<CollisionAlgorithm>());
}
// Reject unknown modes before dispatch through the abstract algorithm interface.
Result Engine::execute(const Scene &scene, Mode mode, bool record) const {
    auto found = algorithms_.find(mode);
    if (found == algorithms_.end())
        throw std::invalid_argument("Unknown algorithm mode.");
    return found->second->run(scene, record);
}
```

The registry contains both DBSCAN variants as differently configured instances of one class. execute() validates the mode lookup and invokes the selected object's common run() method. The virtual calculation inside that method chooses the concrete implementation. No owning raw new and delete pair is required.

C.4 Inheritance with a shared route algorithm

RouteAlgorithm derives from Algorithm and provides the shared calculation. DijkstraAlgorithm and AStarAlgorithm derive from RouteAlgorithm. This is multilevel inheritance with two concrete sibling specializations.

Source `include/algorithms.hpp` lines 90 to 109.

```
class RouteAlgorithm : public Algorithm {
    Result calculate(const Scene &scene, bool record) const final;

  protected:
    virtual double estimate(const Scene &scene, int from, int goal) const = 0;
};
// Route specialization whose heuristic is identically zero.
class DijkstraAlgorithm final : public RouteAlgorithm {
    double estimate(const Scene &, int, int) const override { return 0.0; }

  public:
    Mode mode() const noexcept override { return Mode::Dijkstra; }
};
// Route specialization using Euclidean or grid-Manhattan estimates.
class AStarAlgorithm final : public RouteAlgorithm {
    double estimate(const Scene &scene, int from, int goal) const override;

  public:
    Mode mode() const noexcept override { return Mode::AStar; }
};
```

The route loop is final, so the children change the protected estimate() hook rather than duplicate search, relaxation, trace construction and path reconstruction. Dijkstra returns zero; A* supplies a geometric estimate. Both expose their mode through the inherited public contract.

The design supports substitution while preserving a common cost and validity policy. An incompatible A* estimate is reduced to zero by the shared implementation, which retains shortest-path correctness for that input.

Execution example. App calls AlgorithmRunner::execute. Engine selects an AStarAlgorithm object and invokes Algorithm::run. The inherited RouteAlgorithm::calculate loop calls AStarAlgorithm::estimate through the virtual hook. The checked Result returns through the same interfaces used for Dijkstra.

C.5 Spatial index Strategy and dependency substitution

SpatialIndex defines proximity operations. LinearIndex and KDTree implement this contract with different query methods. The virtual destructor keeps exclusive polymorphic ownership safe.

Source `include/algorithms.hpp` lines 23 to 39.

```
class SpatialIndex {
  public:
    virtual ~SpatialIndex() = default;
    virtual int nearest(Point query, Stats &stats, Trace *trace = nullptr) const = 0;
    virtual std::vector<int> radius(Point query, double distance, Stats &stats,
                                    Trace *trace = nullptr) const = 0;
};
// Exhaustive proximity Strategy and direct DBSCAN neighbourhood provider.
class LinearIndex final : public SpatialIndex {
    std::vector<Point> points_;

  public:
    explicit LinearIndex(std::vector<Point> points) : points_(std::move(points)) {}
    int nearest(Point query, Stats &stats, Trace *trace = nullptr) const override;
    std::vector<int> radius(Point query, double distance, Stats &stats,
                            Trace *trace = nullptr) const override;
};
```

The beginning of DBSCAN's calculation selects an index through the interface. The remaining density expansion calls radius() and is shared by both modes.

Source `src/algorithms.cpp` lines 228 to 238.

```
Result DBSCANAlgorithm::calculate(const Scene &s, bool record) const {
    const bool indexed = indexed_;
    Result r(mode(), record);
    int n = static_cast<int>(s.points().size());
    // -2 unassigned; -1 noise; >=0 cluster. kind: 0 noise, 1 border, 2 core.
    std::vector<int> labels(n, -2), kind(n, 0), visited(n, 0);
    std::unique_ptr<SpatialIndex> index;
    if (indexed)
        index = std::make_unique<KDTree>(s.points());
    else
        index = std::make_unique<LinearIndex>(s.points());
```

This is the Strategy pattern: the clustering operation holds one interchangeable neighbourhood strategy. unique_ptr communicates exclusive ownership; make_unique creates the selected implementation. Sorted neighbour IDs preserve the same deterministic cluster and border assignments across both strategies.

The separation permits a useful comparison: labels should agree on identical input, while query-distance work may differ. Tree construction, reference checking and playback remain separate from that counter comparison.

C.6 Composition value snapshots and RAII

Run contains a Scene and Result by value. Its constructor accepts values and moves them into owned members. Const accessors make completed input and results read-only through the public Run interface, so later live edits cannot modify a historical calculation.

Source `include/trace.hpp` lines 207 to 216.

```
class Run {
    Scene input_;
    Result result_;

  public:
    Run() = default;
    Run(Scene input, Result result) : input_(std::move(input)), result_(std::move(result)) {}
    const Scene &input() const noexcept { return input_; }
    const Result &result() const noexcept { return result_; }
};
```

DrawingSession uses resource acquisition is initialization to balance graphics operations across scope exits. Copy operations are deleted because duplicating the scope owner would cause duplicate cleanup.

Source `src/ui_support.hpp` lines 273 to 287.

```
class DrawingSession final {
  public:
    DrawingSession() {
        BeginDrawing();
        Ui::beginTarget();
        BeginMode2D(Ui::camera());
    }
    ~DrawingSession() {
        EndMode2D();
        Ui::endTarget();
        EndDrawing();
    }
    DrawingSession(const DrawingSession &) = delete;
    DrawingSession &operator=(const DrawingSession &) = delete;
};
```

Construction begins the drawing session; destruction ends it. The Window wrapper similarly pairs InitWindow with CloseWindow, and Ui owns loaded fonts. Standard containers and unique_ptr provide the same lifetime discipline for ordinary data and polymorphic strategies.

Composition describes which object owns another object's lifetime. RAII describes when cleanup happens. Move semantics transfers an owned value efficiently; it is a language mechanism that supports these ownership choices.

C.7 Adapter abstraction and the Memento pattern

The ingestion boundary depends on a LogAdapter interface. CsvLogAdapter supplies the local CSV implementation without adding parser details to the geometry algorithms.

Source `include/security.hpp` lines 124 to 133.

```
class LogAdapter {
  public:
    virtual ~LogAdapter() = default;
    virtual std::vector<Event> load(const std::string &path) const = 0;
};
// Strict single-line CSV implementation of the log-loading interface.
class CsvLogAdapter final : public LogAdapter {
  public:
    std::vector<Event> load(const std::string &path) const override;
};
```

WorkspaceState composes the editable Lab model, security model and unfinished fence. EditHistory retains these values as input mementos.

Source `include/workspace.hpp` lines 8 to 14.

```
struct WorkspaceState {
    Scene lab;
    SecurityConsole security;
    std::vector<Point> fenceDraft;
    WorkspaceState(Scene labInput = {}, SecurityConsole console = {}, std::vector<Point> draft = {})
        : lab(std::move(labInput)), security(std::move(console)), fenceDraft(std::move(draft)) {}
};
```

Source `include/workspace.hpp` lines 84 to 98.

```
    bool record(const WorkspaceState &state) {
        remember({state.lab, state.security, state.fenceDraft});
        auto key = workspaceKey(state);
        if (!states_.empty() && currentKey_ == key)
            return false;
        if (!states_.empty())
            states_.erase(states_.begin() + static_cast<std::ptrdiff_t>(cursor_ + 1),
                          states_.end());
        states_.push_back(state);
        currentKey_ = std::move(key);
        if (states_.size() > 128)
            states_.erase(states_.begin());
        cursor_ = states_.size() - 1;
        return true;
    }
```

record() avoids equivalent duplicate states, removes a redo branch after a new edit and bounds history to 128 states. Undo and redo restore stored input through the public history interface. This editable input history is separate from Run's completed calculation evidence.